

Code Complexity Analysis

1. Pseudocode

Function 1: rightRotation(node)

location: /Tree/Avl.java

```
FUNCTION rightRotation(node):  
    leftChild = node.left  
    node.left = leftChild.right  
    leftChild.right = node  
  
    nodeHeight = 1 + MAX(  
        IF node.left ≠ NULL THEN node.left.element.height ELSE -1,  
        IF node.right ≠ NULL THEN node.right.element.height ELSE -1)  
  
    node.element.height = nodeHeight  
  
    leftHeight = 1 + MAX(  
        IF leftChild.left ≠ NULL THEN leftChild.left.element.height ELSE -1,  
        IF leftChild.right ≠ NULL THEN leftChild.right.element.height ELSE -1)  
  
    leftChild.element.height = leftHeight  
  
    RETURN leftChild  
END FUNCTION
```

Function 2: leftRotation(node)

location: /Tree/Avl.java

```
FUNCTION leftRotation(node):
    rightChild = node.right
    node.right = rightChild.left
    rightChild.left = node

    nodeHeight = 1 + MAX(
        IF node.left ≠ NULL THEN node.left.element.height ELSE -1,
        IF node.right ≠ NULL THEN node.right.element.height ELSE -1)

    node.element.height = nodeHeight

    rightHeight = 1 + MAX(
        IF rightChild.left ≠ NULL THEN rightChild.left.element.height ELSE -1,
        IF rightChild.right ≠ NULL THEN rightChild.right.element.height ELSE -1)

    rightChild.element.height = rightHeight

    RETURN rightChild
END FUNCTION
```

Function 3: twoRotation(node)

location: /Tree/Avl.java

```
FUNCTION twoRotations(node):
    IF balanceFactor(node) > 0:
        node.right = rightRotation(node.right)
        node = leftRotation(node)
    ELSE:
        node.left = leftRotation(node.left)
        node = rightRotation(node)
    RETURN node

END FUNCTION
```

Function 4: balanceNode()

location: /Tree/Avl.java

```

FUNCTION balanceNode(node):
    balance = balanceFactor(node)

    IF balance > 1: // Right-heavy
    IF balanceFactor(node.right) < 0:
        node = twoRotations(node) // Right-Left case
    ELSE:
        node = leftRotation(node) // Right-Right case

    ELSE IF balance < -1: // Left-heavy
    IF balanceFactor(node.left) > 0:
        node = twoRotations(node) // Left-Right case
    ELSE:
        node = rightRotation(node) // Left-Left case

    RETURN node
END FUNCTION

```

Function 5: balanceFactor(node)

location: /Tree/Avl.java

```

FUNCTION balanceFactor(node):
    rightHeight = IF node.right ≠ NULL THEN node.right.element.height ELSE -1
    leftHeight = IF node.left ≠ NULL THEN node.left.element.height ELSE -1

    RETURN rightHeight - leftHeight
END FUNCTION

RETURN FUNCTION
END FUNCTION

```

- **Note:** Heights are retrieved in **O(1)** due to caching in each NodeData.

Function 6: insert()

location: /Tree/Avl.java

```
FUNCTION insert(element):
    root = insertHelper(element, root)
END FUNCTION

FUNCTION insertHelper(element, node):
    IF node IS NULL:
        RETURN new Node(element, NULL, NULL)

    ELSE IF element < node.element:
        node.left = insertHelper(element, node.left)
    ELSE:
        node.right = insertHelper(element, node.right)

    leftHeight = IF node.left ≠ NULL THEN node.left.element.height ELSE -1
    rightHeight = IF node.right ≠ NULL THEN node.right.element.height ELSE -1

    node.element.height = 1 + MAX(leftHeight, rightHeight)

    node = balanceNode(node)

    RETURN node
END FUNCTION
```

Function 7: findNodeByCoordinates()

location: /Tree/Avl.java

```
FUNCTION findNodeByCoordinates(coord):  
    RETURN findNodeByCoordinatesHelper(coord, root)  
END FUNCTION  
  
FUNCTION findNodeByCoordinatesHelper(coord, node):  
    IF node IS NULL:  
        RETURN NULL  
  
    data = node.getElement()  
    cmpLat = compare(coord.latitude, data.coordinate.latitude)  
    IF cmpLat ≠ 0:  
        IF cmpLat < 0:  
            RETURN findNodeByCoordinatesHelper(coord, node.left)  
        ELSE:  
            RETURN findNodeByCoordinatesHelper(coord, node.right)  
  
    cmpLon = compare(coord.longitude, data.coordinate.longitude)  
    IF cmpLon ≠ 0:  
        IF cmpLon < 0:  
            RETURN findNodeByCoordinatesHelper(coord, node.left)  
        ELSE:  
            RETURN findNodeByCoordinatesHelper(coord, node.right)  
  
    RETURN data  
END FUNCTION
```

Function 8: parseEUStations

location: Parser/Parser64k.java

```

FUNCTION parseEUStations():
    avl = new AVL()
    inputStream = openResource("train_stations_europe.csv")
    IF inputStream IS NULL:
        THROW error "File not found"

    reader = BufferedReader(InputStreamReader(inputStream))
    header = reader.readLine() // skip header

    FOR each line IN reader:
        fields = split(line, ",")

    IF length(fields) < 10:
        CONTINUE // skip incomplete rows

    tzGroup = TimeZoneGroup.fromString(fields)[1]
    IF tzGroup IS NULL:
        CONTINUE

    name = fields[2]
    countryCode = fields

    IF name is empty OR countryCode is empty:
        CONTINUE

    location = new GeographicalLocation()
    IF NOT parseCheckLatLong(fields, fields, location):
        CONTINUE

    isCity = (fields == "True")
    isMain = (fields == "True")
    isAirport = (fields == "True")

    station = new Station(location, tzGroup, ..., country, name, ..., isCity, isAirport, isMain)

    node = avl.findNodeByCoordinates(location)
    IF node IS NOT NULL:
        node.addStation(station) // add, keeping name order
    ELSE:
        avl.insert(new NodeData(location, station))

    RETURN avl
END FUNCTION

```

2. Time Complexity

Table: rightRotation Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
Get left child	Access node.left	O(1)	O(1)	O(1)	Pointer access
Reattach left's right	Set node.left = leftChild.right	O(1)	O(1)	O(1)	Pointer assignment
Attach node as rightChild	Set leftChild.right = node	O(1)	O(1)	O(1)	Pointer assignment
Compute nodeHeight	Height update via MAX of children	O(1)	O(1)	O(1)	Heights cached in NodeData
Set node.element.height	Update height in node	O(1)	O(1)	O(1)	Integer assignment
Compute leftHeight	Height update for new root	O(1)	O(1)	O(1)	Heights cached in NodeData
Set leftChild.height	Update height in new root	O(1)	O(1)	O(1)	Integer assignment
Return new root	Return leftChild	O(1)	O(1)	O(1)	Value return
TOTAL	Complete operation (right rotation)	O(1)	O(1)	O(1)	Purely local update, constant time

Table: leftRotation Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
Get right child	Access node.right	O(1)	O(1)	O(1)	Pointer access
Reattach right's left	Set node.right = rightChild.left	O(1)	O(1)	O(1)	Pointer assignment
Attach node as leftChild	Set rightChild.left = node	O(1)	O(1)	O(1)	Pointer assignment
Compute nodeHeight	Height update via MAX of children	O(1)	O(1)	O(1)	Heights cached in NodeData
Set node.element.height	Update height in node	O(1)	O(1)	O(1)	Integer assignment
Compute rightHeight	Height update for new root	O(1)	O(1)	O(1)	Heights cached in NodeData
Set rightChild.height	Update height in new root	O(1)	O(1)	O(1)	Integer assignment
Return new root	Return rightChild	O(1)	O(1)	O(1)	Value return
TOTAL	Complete operation (left rotation)	O(1)	O(1)	O(1)	Purely local update, constant time

Table: twoRotations Function

Operation	Description	Best Case	Average Case	Worst Case
Compute balance factor	Call balanceFactor(node)	O(1)	O(1)	O(1)
Right-Left case:				
- Right rotation on right child	node.setRight(rightRotation(node.getRight()))	O(1)	O(1)	O(1)
- Left rotation on node	node = leftRotation(node)	O(1)	O(1)	O(1)

Operation	Description	Best Case	Average Case	Worst Case
Left-Right case:				
- Left rotation on left child	node.setLeft(leftRotation(node.getLeft()))	O(1)	O(1)	O(1)
- Right rotation on node	node = rightRotation(node)	O(1)	O(1)	O(1)
TOTAL	Complete operation (double rotation)	O(1)	O(1)	O(1)

Table: balanceNode Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
Compute balance factor	Calculate balanceFactor(node)	O(1)	O(1)	O(1)	Height is cached
Check tree balance	Compare balance to 1 or -1	O(1)	O(1)	O(1)	Simple integer check
Branch to case (rotation)	Select appropriate rotation(s)	O(1)	O(1)	O(1)	At most two nested balanceFactor calls
Perform single/double rotate	Call leftRotation, rightRotation, or twoRotations	O(1)	O(1)	O(1)	Each is local and O(1), already analyzed
Return new subtree root	Return node after rotation	O(1)	O(1)	O(1)	Pointer return
TOTAL	Complete operation (one rebalance)	O(1)	O(1)	O(1)	Only direct children/rotation, constant time

Table: balanceFactor Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
Access right height	Get node.right.element.height or -1 if null	O(1)	O(1)	O(1)	Heights cached in each node
Access left height	Get node.left.element.height or -1 if null	O(1)	O(1)	O(1)	Heights cached in each node
Subtract heights	Compute difference (right - left)	O(1)	O(1)	O(1)	Simple integer operation
TOTAL	Calculate balance factor for a node	O(1)	O(1)	O(1)	No recursion, single node computation

Table: insert Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
Insert traversal	Recursively descend (BST logic)	O(log n)	O(log n)	O(n)	O(log n) if tree balanced, O(n) if skewed
Compare to node.element	Determine insertion side	O(1)	O(1)	O(1)	Per comparison
Insert at null	Create new node	O(1)	O(1)	O(1)	Leaf node creation
Update heights	Compute new height from children's heights	O(1)	O(1)	O(1)	Cached in each NodeData

Operation	Description	Best Case	Average Case	Worst Case	Notes
Balance on way up	Call balanceNode at most once per ancestor	$O(1)$	$O(1)$	$O(1)$	Each rebalance is $O(1)$; total recursive calls $O(\log n)$
Pointer/field assignments	Link children or update element	$O(1)$	$O(1)$	$O(1)$	
TOTAL (per insert)	Complete AVL insert for one element	$O(\log n)$	$O(\log n)$	$O(n)$	Balanced: $O(\log n)$; Only unbalanced (highly unlikely) is $O(n)$

Table: findNodeByCoordinates Function

Operation	Description	Best Case	Average Case	Worst Case	Notes
Compare latitude	Compare search coordinate with node's latitude	$O(1)$	$O(1)$	$O(1)$	Simple numeric comparison
Traverse left or right child	Recursive call based on comparison	$O(1)$ per call	$O(1)$ per call	$O(1)$ per call	At most one child visited per recursive step
Compare longitude	If latitudes equal, compare longitudes	$O(1)$	$O(1)$	$O(1)$	Simple numeric comparison
Recursive search	Continue down the tree until leaf or match	$O(\log n)$	$O(\log n)$	$O(n)$	Balanced tree: height $\approx \log n$; skewed worst case $O(n)$
Return matching node	Return found node or NULL	$O(1)$	$O(1)$	$O(1)$	Constant time
TOTAL	Complete coordinate search operation	$O(\log n)$	$O(\log n)$	$O(n)$	Balanced AVL tree ensures logarithmic search time

Table: parseEUStations Function

Let:

- (N) = number of valid lines (stations) processed from the CSV
- (n) = number of distinct coordinates (AVL nodes) inserted into the AVL
- (k) = number of stations at a given coordinate (almost always very small; treated as $O(1)$)

Operation	Description	Best Case	Average Case	Worst Case	Notes
Open resource & reader	Set up input stream and buffered reader	$O(1)$	$O(1)$	$O(1)$	One-time initialization
Read & split line	readLine + split	$O(1)$ per line	$O(1)$ per line	$O(1)$ per line	Simple string operations
Validation & parse fields	Validate field count, parse objects	$O(1)$	$O(1)$	$O(1)$	Constant number of checks
parseCheckLatLong	Validate/set latitude & longitude	$O(1)$	$O(1)$	$O(1)$	Fast reject for invalid coordinates
findNodeByCoordinates	Search AVL by (lat, lon)	$O(\log n)$	$O(\log n)$	$O(n)$	Balanced: $O(\log n)$; degenerate (rare): $O(n)$

Operation	Description	Best Case	Average Case	Worst Case	Notes
Add station to node (if found)	Insert into sorted list at node	$O(1)$ – $O(k)$	$O(1)$	$O(k)$	(k) is small—effectively $O(1)$ in real data
Insert new node into AVL	AVL insert + update heights/balance	$O(\log n)$	$O(\log n)$	$O(n)$	Balanced: $O(\log n)$; pointer updates and height $O(1)$
Loop over all lines	Entire CSV file of (N) lines	$O(N)$	$O(N)$	$O(N)$	Each line is handled independently
TOTAL	Parse + insert all rows into AVL	$O(N \log n)$	$O(N \cdot \log n)$	$O(N \cdot n)$	Worst case only happens if tree is unbalanced (rotation in every insert)